

TABLE 8-1 Encoding of Register Selection Fields

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

TABLE 8-2 Encoding of ALU Operations

OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	Add $A + B$	ADD
00101	Subtract $A - B$	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA



Examples of Microoperations for the CPU

TABLE 8-3 Examples of Microoperations for the CPU

Microoperation	Symbolic Designation				Control Word
	SELA	SELB	SELD	OPR	
$R1 \leftarrow R2 - R3$	R2	R3	R1	SUB	010 011 001 00101
$R4 \leftarrow R4 \vee R5$	R4	R5	R4	OR	100 101 100 01010
$R6 \leftarrow R6 + 1$	R6	—	R6	INCA	110 000 110 00001
$R7 \leftarrow R1$	R1	—	R7	TSFA	001 000 111 00000
$\text{Output} \leftarrow R2$	R2	—	None	TSFA	010 000 000 00000
$\text{Output} \leftarrow \text{Input}$	Input	—	None	TSFA	000 000 000 00000
$R4 \leftarrow \text{shl } R4$	R4	—	R4	SHLA	100 000 100 11000
$R5 \leftarrow 0$	R5	R5	R5	XOR	101 101 101 01100



- 8-3 Stack Organization
 - Stack or LIFO (*Last-In, First-Out*)
 - A storage device that stores information
 - The item stored last is the first item retrieved = a stack of tray
 - Stack Pointer (**SP**)
 - The register that holds the address for the stack
 - SP always points at the top item in the stack
 - Two Operations of a stack : *Insertion and Deletion of Items*
 - PUSH : Push-Down = Insertion
 - POP : Pop-Up = Deletion
 - Stack
 - 1) Register Stack (*Stack Depth*)
 - a finite number of memory words or register(*stand alone*)
 - 2) Memory Stack (*Stack Depth*)
 - a portion of a large memory
 - Register Stack : **Fig. 8-3**

- PUSH :

* SP = 0,
EMTY = 1,
FULL = 0

$$SP \leftarrow SP + 1$$

: Increment SP

$$M[SP] \leftarrow DR$$

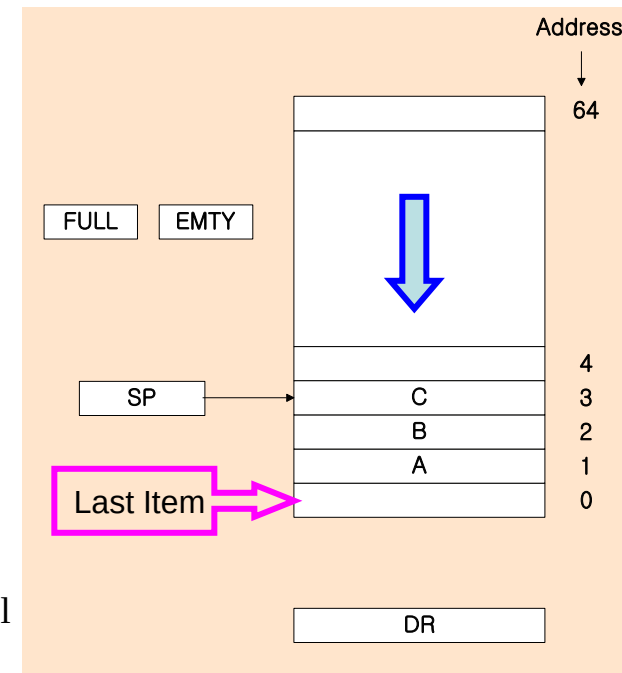
: Write to the stack

$$\text{If } (SP = 0) \text{ then } (FULL \leftarrow 1)$$

: Check if stack is full

$$EMTY \leftarrow 0$$

: Mark not empty



- The first item is stored at *address 1*, and the last item is stored at *address 0*

- POP : $DR \leftarrow M[SP]$: Read item from the top of stack
 $SP \leftarrow SP - 1$: Decrement Stack Pointer
If ($SP = 0$) *then* ($EMPTY \leftarrow 1$) : Check if stack is empty
 $FULL \leftarrow 0$: Mark not full

* Memory Stack
 PUSH = Address
 * Register Stack
 PUSH = Address

- Memory Stack : *Fig. 8-4*

- PUSH : $SP \leftarrow SP - 1$
 $M[SP] \leftarrow DR$

* $SP = 4001$ the first item is stored at *address 4000*

- POP : $DR \leftarrow M[SP]$
 $SP \leftarrow SP + 1$

* Error Condition
 PUSH when $FULL = 1$
 POP when $EMPTY = 1$

- Stack Limits

- Check for stack overflow(*full*)/underflow(*empty*)

- Checked by using two register
 - Upper Limit and Lower Limit Register
- After PUSH Operation
 - » SP compared with the upper limit register
- After POP Operation
 - » SP compared with the lower limit register

